

Lab Taks-2

Submission Guidelines-

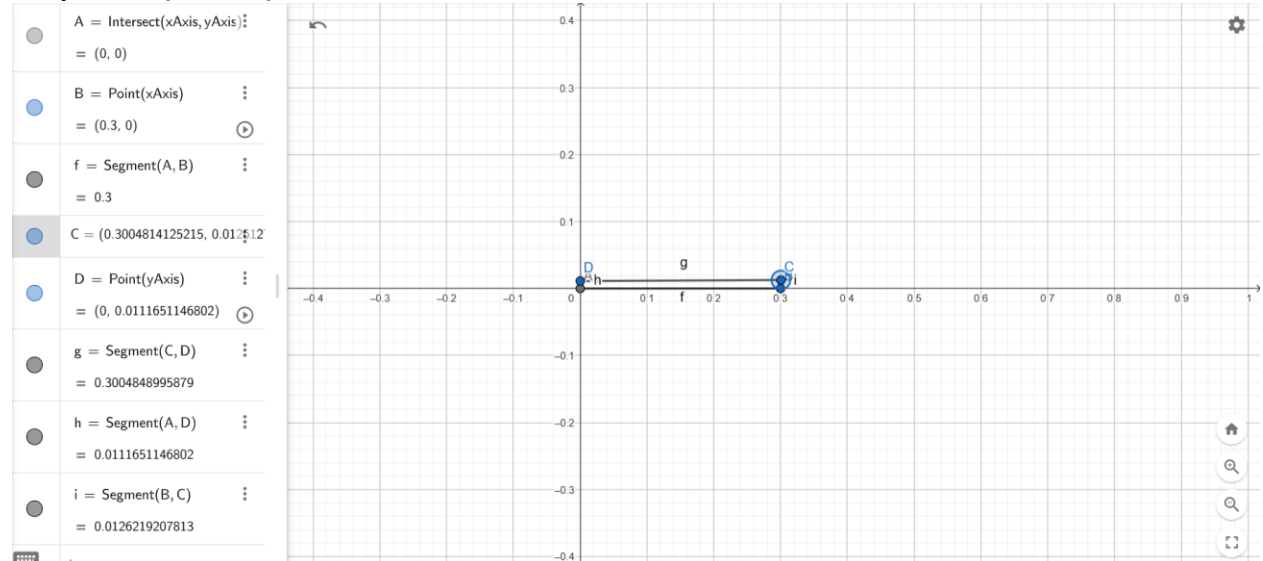
- Rename the file with your serial number only.
- Must submit within time that will be discussed in class VUES to the section named Lab Tak-2
- Must include resources for all the section in the table

Question- 1

Draw a Rainbow Flag



Graph Plot (Picture)-



Code- #include <GL/glut.h>

```
float width = 0.3f;  
float height = 0.12f;
```

```

int numStripes = 6;

// Rainbow colors (RGB)
float colors[6][3] = {
    {228/255.0f, 3/255.0f, 3/255.0f}, // Red
    {255/255.0f, 140/255.0f, 0/255.0f}, // Orange
    {255/255.0f, 237/255.0f, 0/255.0f}, // Yellow
    {0/255.0f, 128/255.0f, 38/255.0f}, // Green
    {0/255.0f, 77/255.0f, 255/255.0f}, // Blue
    {117/255.0f, 7/255.0f, 135/255.0f} // Violet
};

void drawRectangle(float x, float y, float width, float height, float color[3]) {
    glColor3fv(color);
    glBegin(GL_QUADS);
    glVertex2f(x, y);
    glVertex2f(x + width, y);
    glVertex2f(x + width, y + height);
    glVertex2f(x, y + height);
    glEnd();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    float stripeHeight = height / numStripes;
    for (int i = 0; i < numStripes; i++) {
        float y = i * stripeHeight;
        drawRectangle(0, y, width, stripeHeight, colors[i]);
    }
    glColor3f(0, 0, 0);
    glLineWidth(2);
    glBegin(GL_LINE_LOOP);
    glVertex2f(0, 0);
    glVertex2f(width, 0);
    glVertex2f(width, height);
    glVertex2f(0, height);
    glEnd();

    glFlush();
}

void init() {
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f); // White background
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, 0.3, 0.0, 0.12); // Set the coordinate system
}

int main(int argc, char** argv) {

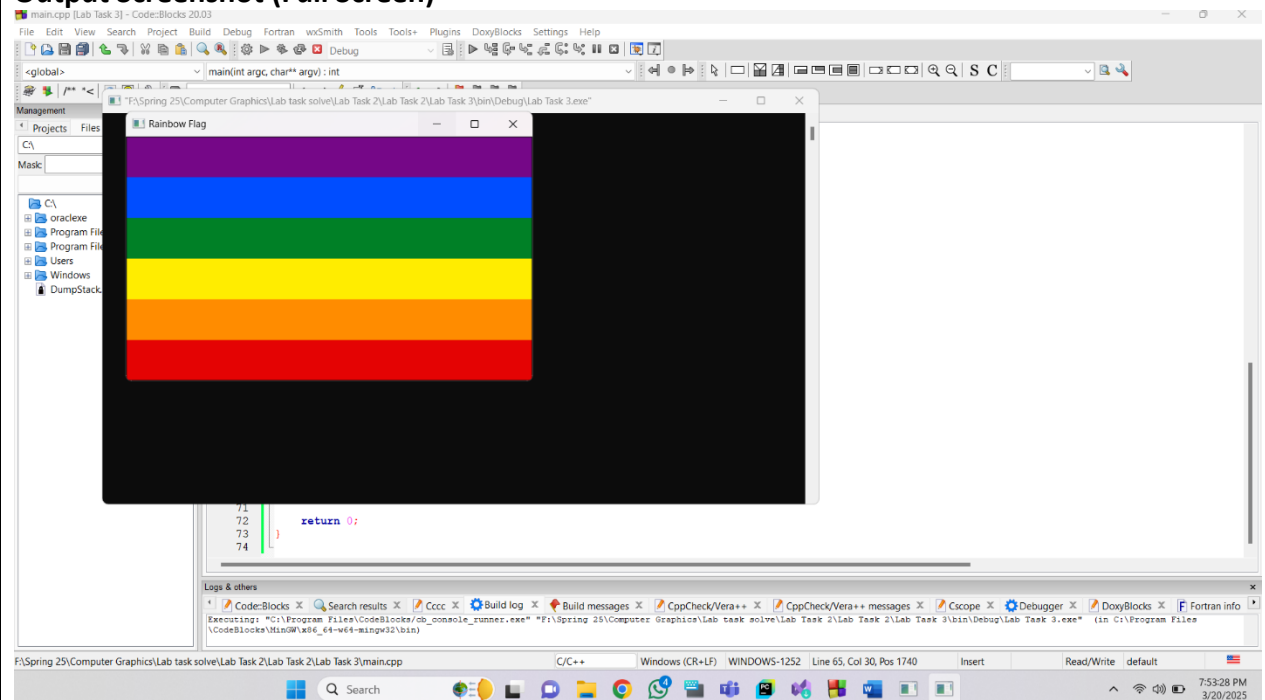
```

```
glutInit(&argc, argv);
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
glutInitWindowSize(500,300);
glutCreateWindow("Rainbow Flag");

init();
glutDisplayFunc(display);
glutMainLoop();

return 0;
}
```

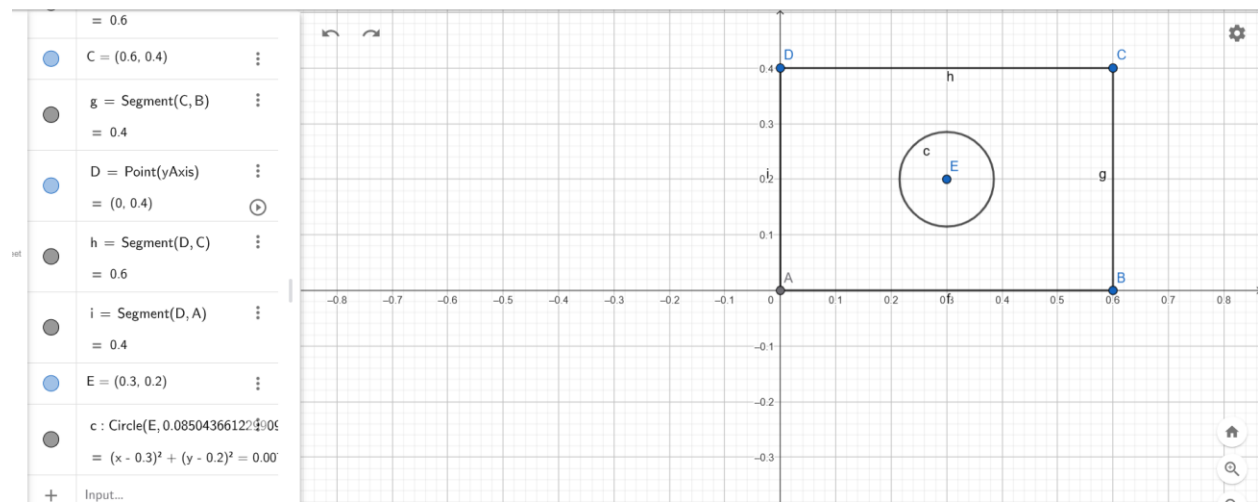
Output Screenshot (Full Screen)-



Question- 2

Flag of 5 different countries as per your choice.

Graph Plot (Picture)-



Code-

BANGLADESH FLAG

```
#include <GL/glut.h>
```

```
#include <cmath>
```

```
// Define flag dimensions
```

```
float flagWidth = 0.6f;
```

```
float flagHeight = 0.4f;
```

```
float circleRadius = 0.085f; // Adjusted for proper scaling
```

```
float circleCenterX = 0.3f; // Center of the circle
```

```
float circleCenterY = 0.2f; // Center of the circle
```

```
// Function to draw a filled circle
```

```
void drawCircle(float cx, float cy, float r, int numSegments) {
```

```
    glBegin(GL_TRIANGLE_FAN);
```

```
    glVertex2f(cx, cy);
```

```
    for (int i = 0; i <= numSegments; i++) {
```

```
        float angle = 2.0f * 3.1416f * float(i) / float(numSegments);
```

```
        float x = r * cos(angle);
```

```

        float y = r * sin(angle);
        glVertex2f(cx + x, cy + y);
    }
    glEnd();
}

// Display function
void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw the green rectangle (flag background)
    glColor3f(0.0f, 0.42f, 0.24f); // Dark green color
    glBegin(GL_QUADS);
    glVertex2f(0.0f, 0.0f);
    glVertex2f(flagWidth, 0.0f);
    glVertex2f(flagWidth, flagHeight);
    glVertex2f(0.0f, flagHeight);
    glEnd();

    // Draw the red circle
    glColor3f(0.84f, 0.0f, 0.0f); // Red color
    drawCircle(circleCenterX, circleCenterY, circleRadius, 100);

    glFlush();
}

// Initialize OpenGL
void init() {
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f); // White background
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, 0.6, 0.0, 0.4); // Set the coordinate system
}

// Main function
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 400);
    glutCreateWindow("Bangladesh Flag");

    init();
    glutDisplayFunc(display);
    glutMainLoop();
}

```

```
    return 0;
}
```

Gayana Flag

```
//Guyana
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif

#include <stdlib.h>

void display()
{
    glClearColor(1.0f,1.0f,1.0f,1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(1);

    glBegin(GL_QUADS);
    glColor3f(0.0f,0.592f,0.22f);
    glVertex2f(0,0);
    glVertex2f(400,0);
    glVertex2f(480,240);
    glVertex2f(0,240);
    glEnd();

    glBegin(GL_TRIANGLES);
    glColor3f(1,1,1);
    glVertex2f(0,0);
    glVertex2f(400,120);
    glVertex2f(0,240);

    glEnd();

    glBegin(GL_TRIANGLES);
    glColor3f(1,0.82,0);
    glVertex2f(0,15);
    glVertex2f(360,120);
    glVertex2f(0,225);
```

```

glEnd();

    glBegin(GL_TRIANGLES);
    glColor3f(0,0,0);
    glVertex2f(0,0);
    glVertex2f(200,120);
    glVertex2f(0,240);

    glEnd();

    glBegin(GL_TRIANGLES);
    glColor3f(0.93,0.2,0.25);
    glVertex2f(0,15);
    glVertex2f(160,120);
    glVertex2f(0,225);

    glEnd();

glFlush();

}
void myInit (void){
    glClearColor(0.5, 0.5, 0.5, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    //glPointSize(4.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, 400.0, 0.0, 240.0);
}
int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize (400, 240);

```

```
glutInitWindowPosition (80, 30);
glutCreateWindow ("my first attempt");
glutDisplayFunc(display);
myInit ();
glutMainLoop();
}
```

Kuwait Flag

```
//Kuwait
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif

#include <stdlib.h>

void display()
{
    glClearColor(1.0f,1.0f,1.0f,1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(1);

    glBegin(GL_QUADS);
    glColor3f(0.807f,0.0667f,0.149f);
    glVertex2f(0,0);
    glVertex2f(480,0);
    glVertex2f(480,80);
    glVertex2f(0,80);

    glEnd();

    glBegin(GL_QUADS);
    glColor3f(1.0f,1.0f,1.0f);
    glVertex2f(0,80);
    glVertex2f(480,80);
    glVertex2f(480,160);
    glVertex2f(0,160);
}
```



```

    glEnd();

    glBegin(GL_QUADS);
    glColor3f(0.0f,0.478f,0.239f);
    glVertex2f(0,160);
    glVertex2f(480,160);
    glVertex2f(480,240);
    glVertex2f(0,240);

    glEnd();

    glBegin(GL_QUADS);
    glColor3f(0.0f,0.0f,0.0f);
    glVertex2f(0,0);
    glVertex2f(120,80);
    glVertex2f(120,160);
    glVertex2f(0,240);

    glEnd();

    glFlush();
}
void myInIt (void){
    glClearColor(0.5, 0.5, 0.5, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    //glPointSize(4.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, 480.0, 0.0, 240.0);
}
int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize (430, 200);
    glutInitWindowPosition (80, 30);

```

```
glutCreateWindow ("my first attempt");
glutDisplayFunc(display);
myInit ();
glutMainLoop();
}
```

Norway Flag

```
//Norway
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif

#include <stdlib.h>

void display()
{
    glClearColor(1.0f,1.0f,1.0f,1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(1);

    glBegin(GL_QUADS);
    glColor3f(0.729f,0.047f,0.184f);
    glVertex2f(0,0);
    glVertex2f(880,0);
    glVertex2f(880,640);
    glVertex2f(0,640);

    glEnd();

    glBegin(GL_QUADS);
    glColor3f(1.0f,1.0f,1.0f);
    glVertex2f(0,240);
    glVertex2f(880,240);
    glVertex2f(880,400);
    glVertex2f(0,400);

    glEnd();
```

```
glBegin(GL_QUADS);  
glColor3f(1.0f,1.0f,1.0f);  
glVertex2f(240,0);  
glVertex2f(400,0);  
glVertex2f(400,640);  
glVertex2f(240,640);
```

```
glEnd();
```

```
glBegin(GL_QUADS);  
glColor3f(0.0f,0.125f,0.356f);  
glVertex2f(280,0);  
glVertex2f(360,0);  
glVertex2f(360,640);  
glVertex2f(280,640);
```

```
glEnd();
```

```
glBegin(GL_QUADS);  
glColor3f(0.0f,0.125f,0.356f);  
glVertex2f(0,280);  
glVertex2f(880,280);  
glVertex2f(880,360);  
glVertex2f(0,360);
```

```
glEnd();
```

```
glFlush();
```

```
}
```

```

void myInit (void){
    glClearColor(0.5, 0.5, 0.5, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    //glPointSize(4.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, 880.0, 0.0, 640.0);
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize (400, 250);
    glutInitWindowPosition (80, 30);
    glutCreateWindow ("my first attempt");
    glutDisplayFunc(display);
    myInit ();
    glutMainLoop();
}

```

Franch Flag

```
#include <GL/glut.h>
```

```

float flagWidth = 0.6f;
float flagHeight = 0.4f;

```

```

void drawRectangle(float x, float y, float width, float height, float
color[3]) {
    glColor3fv(color);
    glBegin(GL_QUADS);
    glVertex2f(x, y);
    glVertex2f(x + width, y);
    glVertex2f(x + width, y + height);
    glVertex2f(x, y + height);
    glEnd();
}

```

```
void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    float blue[] = {0.0f, 0.0f, 1.0f};
    float white[] = {1.0f, 1.0f, 1.0f};
    float red[] = {1.0f, 0.0f, 0.0f};

    float stripeWidth = flagWidth / 3;
    drawRectangle(0.0f, 0.0f, stripeWidth, flagHeight, blue);
    drawRectangle(stripeWidth, 0.0f, stripeWidth, flagHeight, white);
    drawRectangle(2 * stripeWidth, 0.0f, stripeWidth, flagHeight, red);

    glFlush();
}

void init() {
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, 0.6, 0.0, 0.4);
}

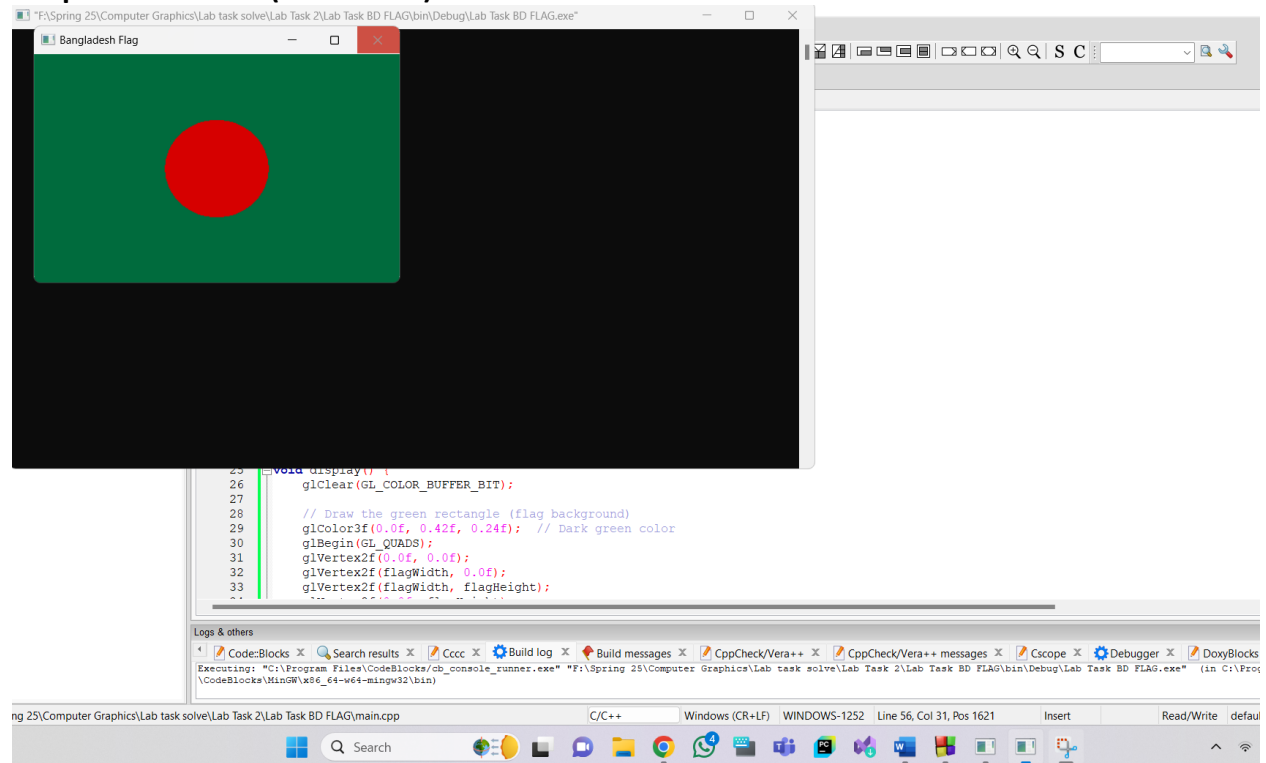
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(400, 250);
    glutCreateWindow("French Flag");

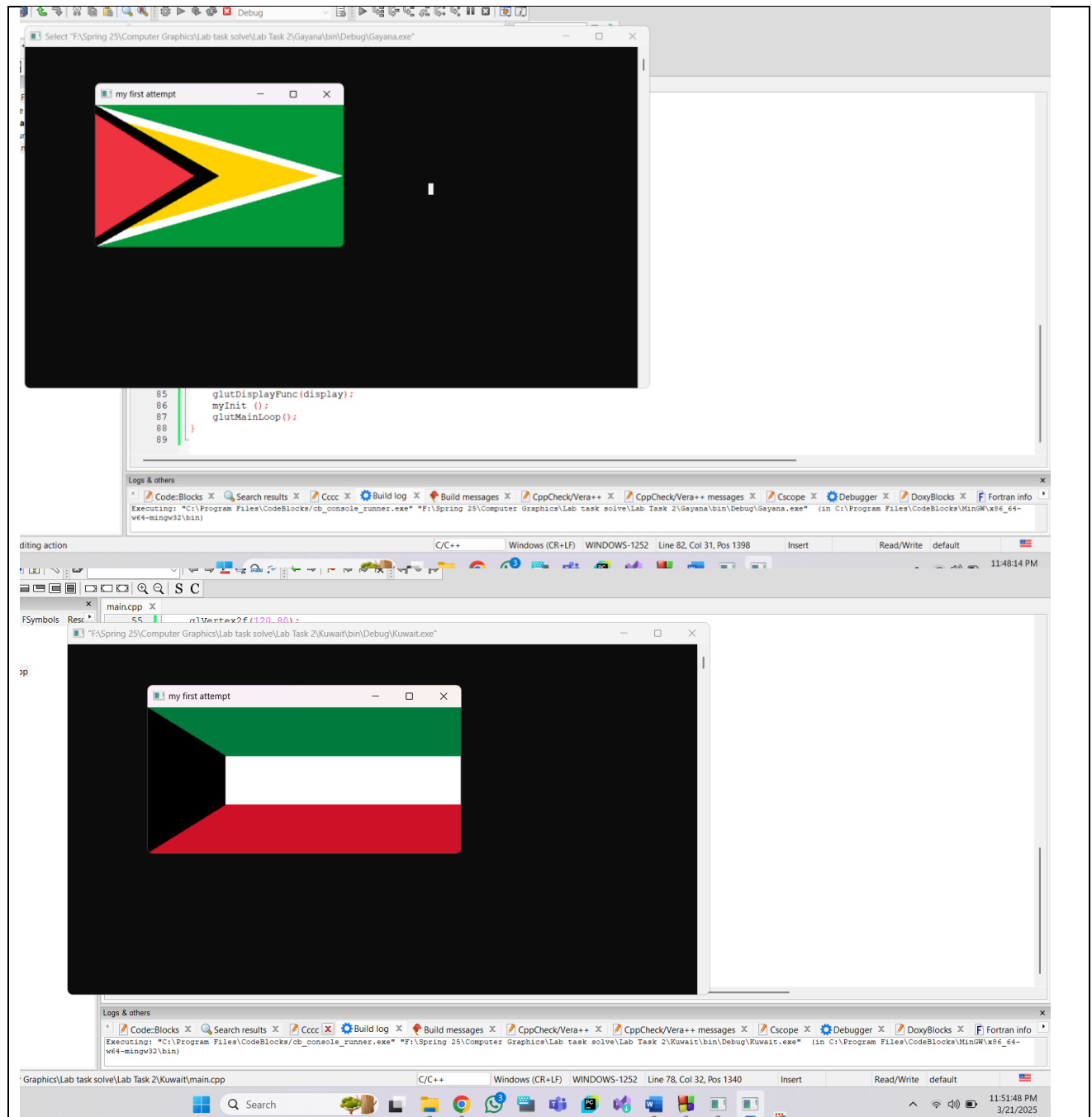
    init();
```

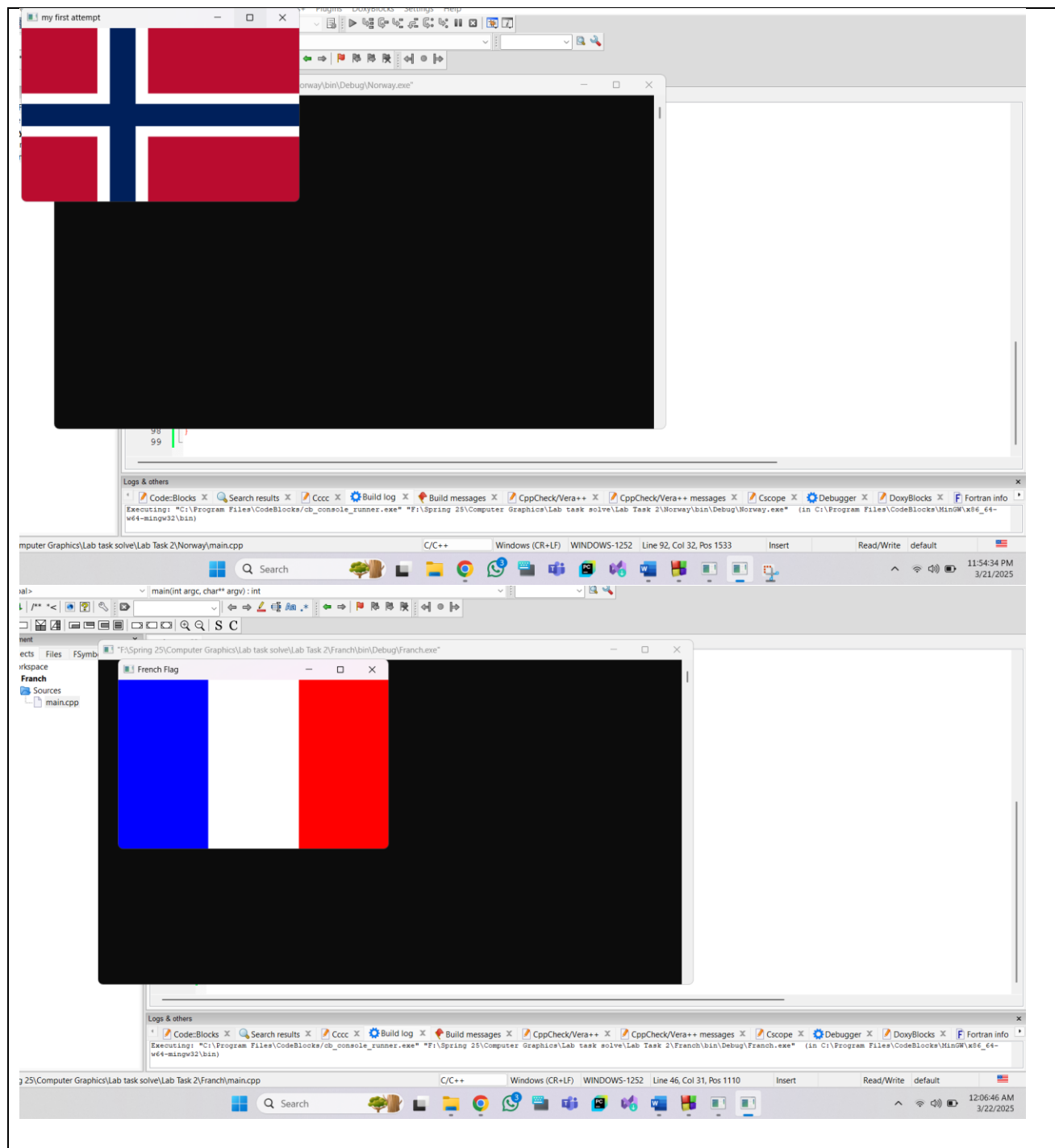
```
glutDisplayFunc(display);  
glutMainLoop();
```

```
return 0;  
}
```

Output Screenshot (Full Screen)-

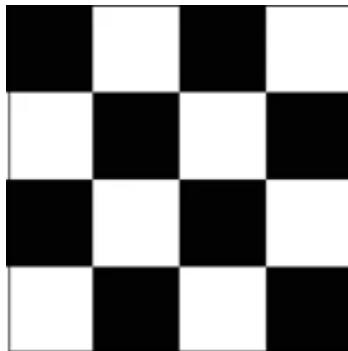




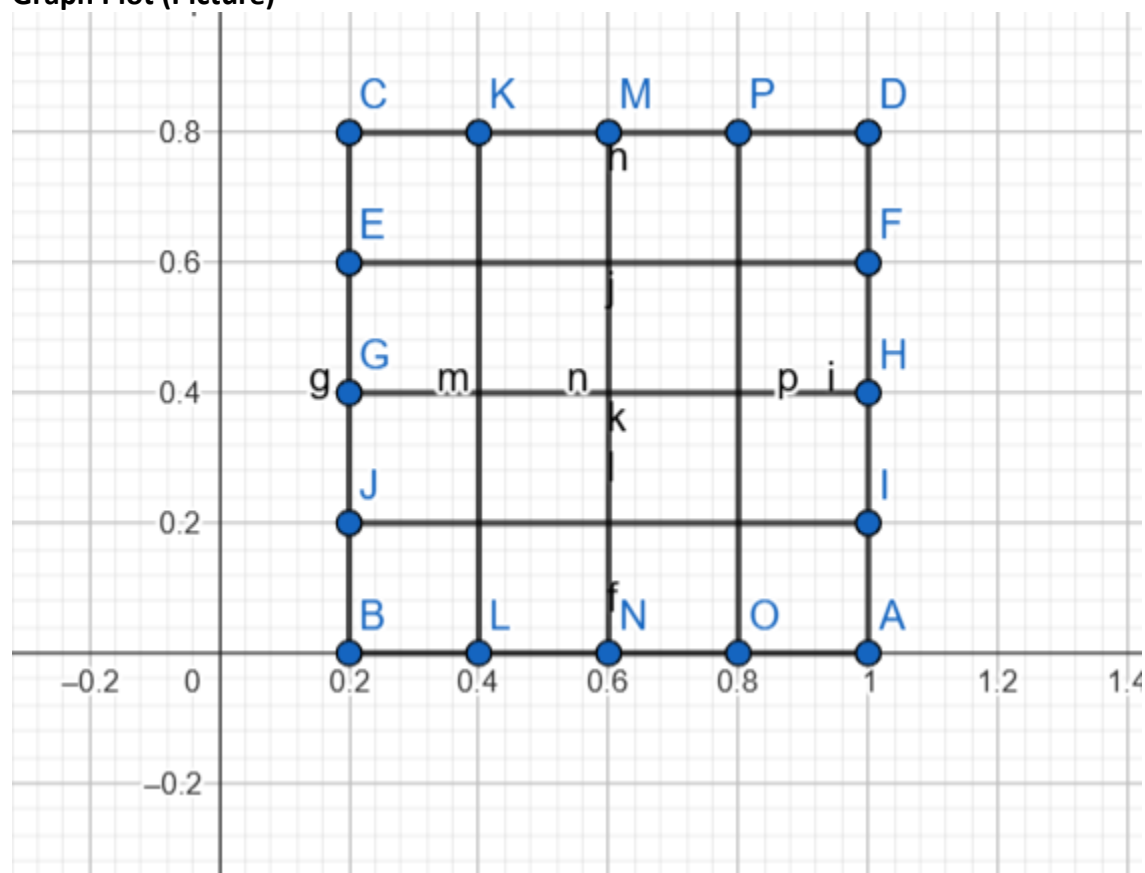


Question- 2

Draw 4X4 Chess Board



Graph Plot (Picture)-



Code-

```
#include <GL/glut.h>
```

```

void drawSquare(float x, float y, float size, bool isBlack) {
    if (isBlack) glColor3f(0.0f, 0.0f, 0.0f); // Black square
    else glColor3f(1.0f, 1.0f, 1.0f); // White square

    glBegin(GL_QUADS);
        glVertex2f(x, y);
        glVertex2f(x + size, y);
        glVertex2f(x + size, y + size);
        glVertex2f(x, y + size);
    glEnd();
}

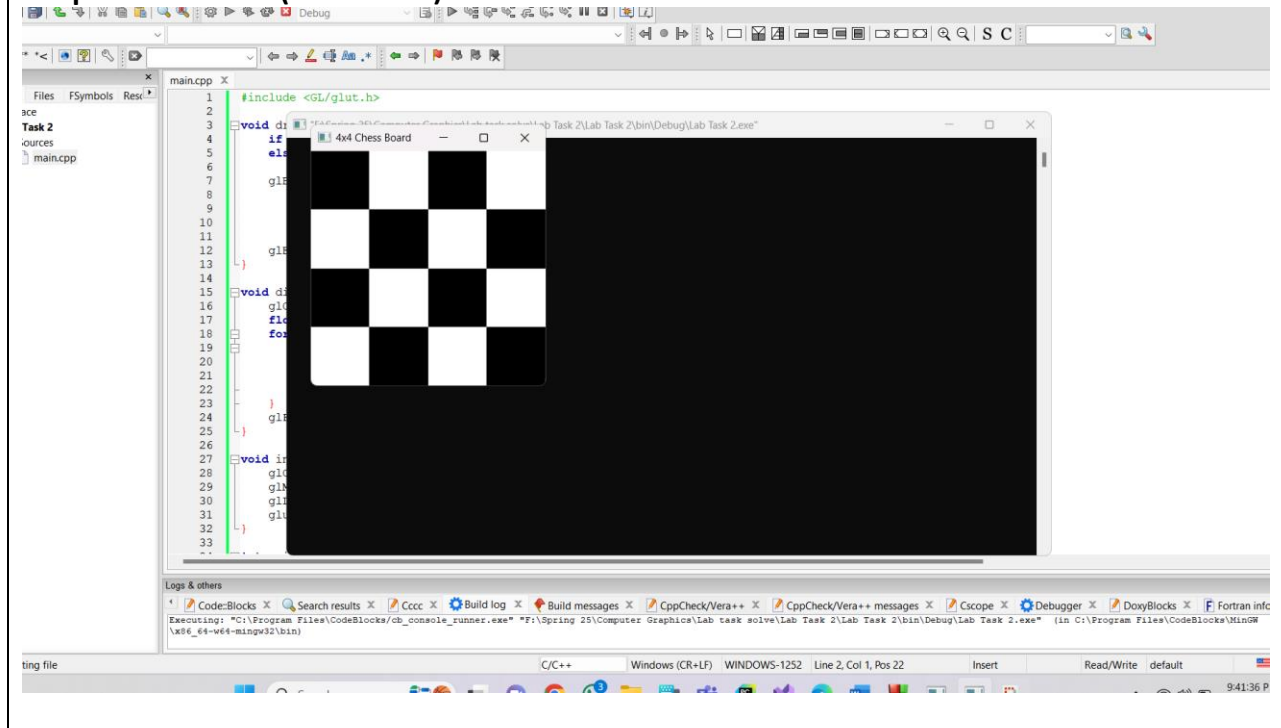
void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    float size = 0.5f; // Size of each square
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            bool isBlack = (i + j) % 2 == 0;
            drawSquare(-1.0f + j * size, 1.0f - (i + 1) * size, size, isBlack);
        }
    }
    glFlush();
}

void init() {
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1.0, 1.0, -1.0, 1.0);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(270, 270);
    glutCreateWindow("4x4 Chess Board");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

Output Screenshot (Full Screen)-



Question- 3

Implement the given picture



Graph Plot (Picture)-

Code-

```
#include <windows.h>
#include <GL/glut.h>
#include <math.h>
```

```

void circle(float r, float g, float b, float radius, float xc, float yc)
{
    glBegin(GL_POLYGON); // Draw a Red 1x1 Square centered at origin
        for(int i=0; i<200; i++)
        {
            glColor3ub(r,g,b);
            float pi=3.1416;
            float A=(i*2*pi)/200;
            float r=radius;
            float x = r * cos(A);
            float y = r * sin(A);
            glVertex2f(x+xc,y+yc);
        }

    //glVertex2f(0.3f,0.4f);
    //glVertex2f(0.1f,0.4f);

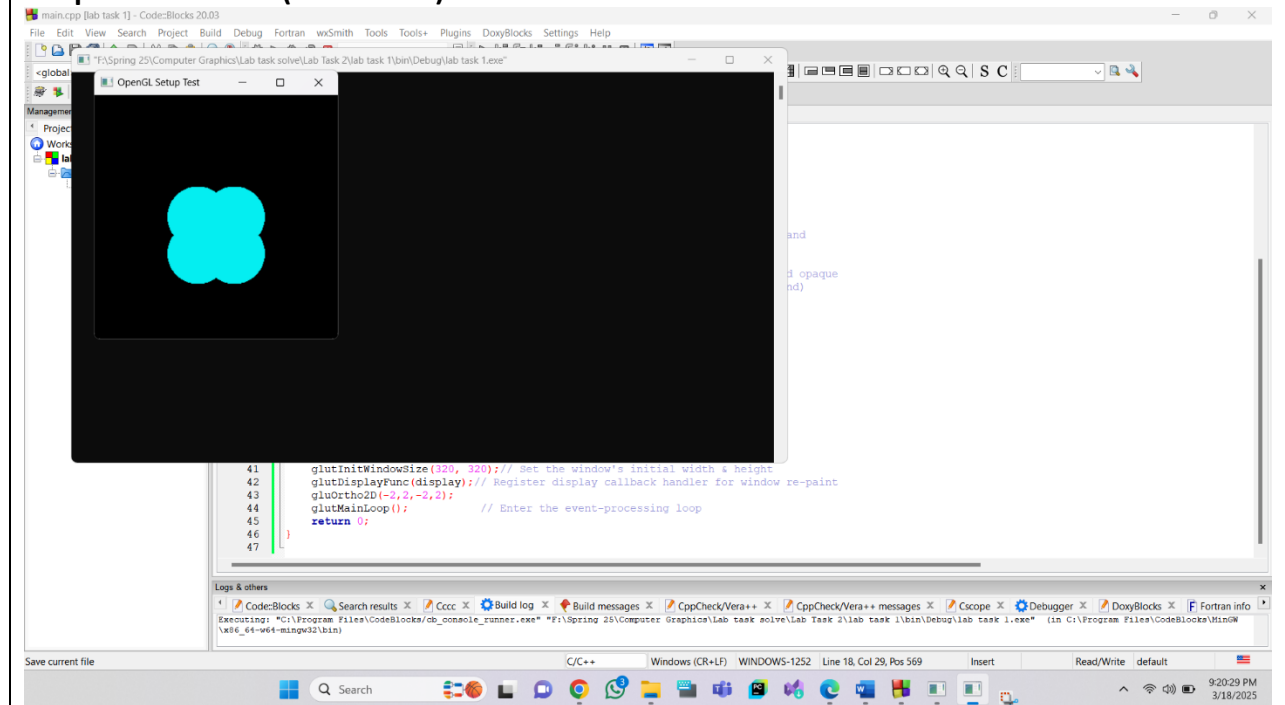
    glEnd();
}

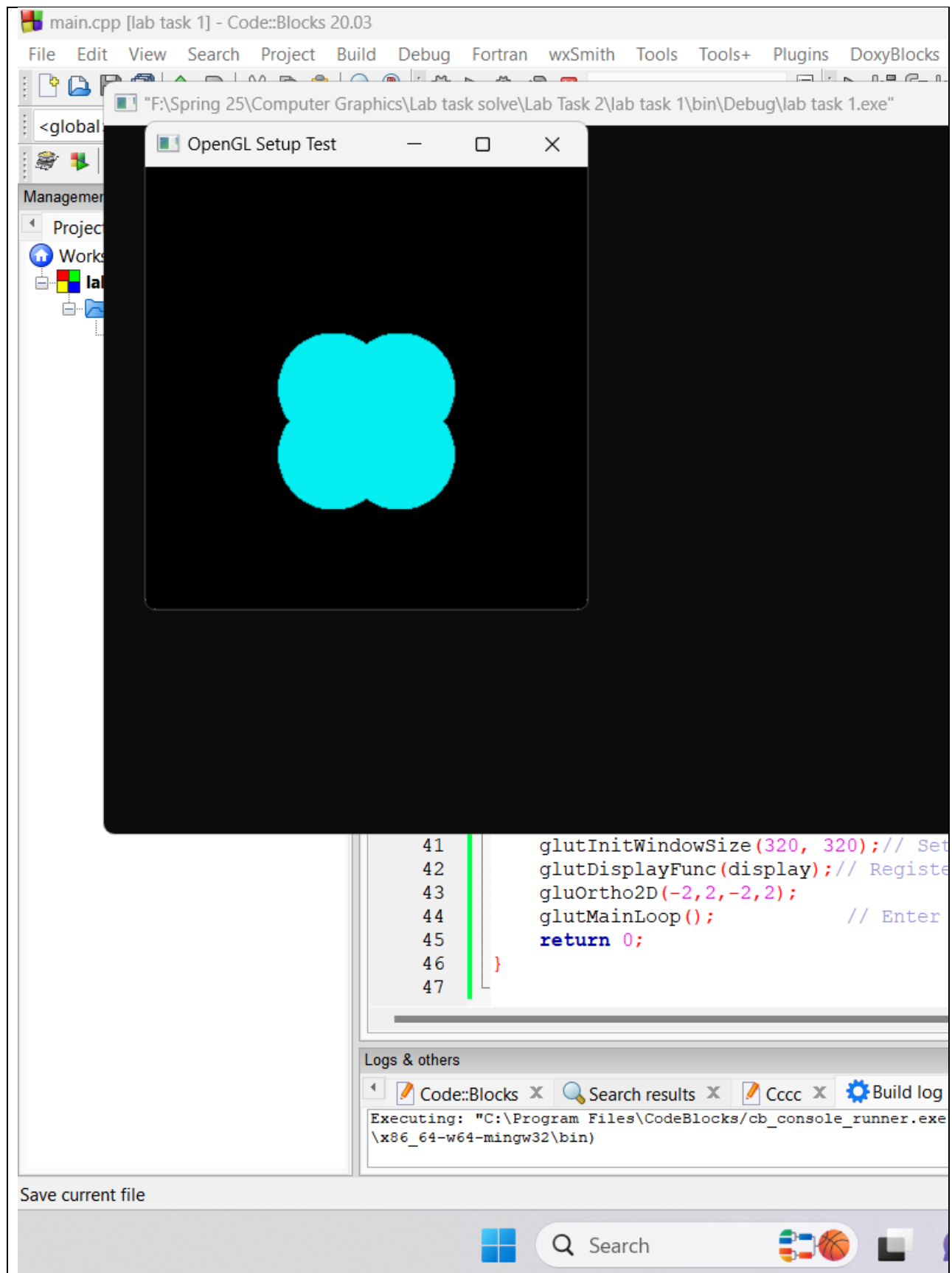
void display() {
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(7.5);
    circle(4, 238, 241, 0.5, -0.3, 0);
    circle(4, 238, 241, 0.5, 0.3, 0);
    circle(4, 238, 241, 0.5, -0.3, -0.6);
    circle(4, 238, 241, 0.5, 0.3, -0.6);
    glFlush(); // Render now
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutCreateWindow("OpenGL Setup Test");
    //gluOrtho2D(-0.1,0.7,-0.1,0.3);
    glutInitWindowSize(320, 320);
    glutDisplayFunc(display);
    gluOrtho2D(-2,2,-2,2);
    glutMainLoop();
    return 0;
}

```

Output Screenshot (Full Screen)-





--